ASSIGNMENT- 03

Bachelor of Computer Applications (Hons. AI & ML)
Semester- III

Subject: Database Management System

Topic: Database Design Analysis

By
Name: Mohammed Sinan nm
Reg. No.: 2511021240024

Faculty Name: RENITA JOY
Faculty Signature:_____

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106
August 2026

DATABASE MANAGEMENT SYSTEM ASSIGNMENT

Database Design Analysis for an Online Food Delivery System

1. Introduction

A database is an essential component of an online food delivery system because it stores and manages information about customers, restaurants, food items, orders, payments, delivery agents, and customer reviews.

In Assignment 1, I designed a database for an Online Food Delivery System similar to Swiggy or Zomato. The design consists of ten tables: Customers, Addresses, Restaurants, MenuItems, Orders, OrderItems, DeliveryAgents, Deliveries, Payments, and Reviews.

The purpose of this assignment is to analyze the existing database design to identify data redundancy, functional dependencies, and possible insertion, deletion, and update anomalies. It also proposes improvements to make the database more efficient, consistent, and reliable.

2. Existing Database Design

The existing database contains the following ten tables:

Sl. No.	Table Name	Purpose
1	Customers	Stores customer information.
2	Addresses	Stores the delivery addresses of customers.
3	Restaurants	Stores restaurant details.
4	MenuItems	Stores food items, prices, and categories.
5	Orders	Stores customer order details.
6	OrderItems	Stores the food items included in each order.
7	DeliveryAgents	Stores delivery agent information.
8	Deliveries	Tracks delivery assignments and delivery status.
9	Payments	Stores payment details for orders.

10	Reviews	Stores customer ratings and feedback.

Each table has a specific responsibility. Primary keys uniquely identify records, while foreign keys establish relationships between related tables.

For example, the customer_id in the Orders table references the Customers table, and the restaurant_id in the MenuItems table references the Restaurants table. The OrderItems table connects Orders and MenuItems, allowing an order to contain multiple food items.

This separation reduces unnecessary repetition and helps maintain data consistency.

3. Identification of Data Redundancy

Data redundancy occurs when the same information is unnecessarily stored multiple times in a database. It increases storage requirements and may cause inconsistencies when the data is modified.

The existing database design reduces redundancy by separating customer details, restaurant information, menu items, and other entities into individual tables. However, some potential redundancy and consistency issues may still occur.

3.1 Customer Information

Customer details such as name, email, and phone number are stored in the Customers table.

If the customer's name and phone number were also stored repeatedly in every order record, the same information would be duplicated whenever the customer placed a new order.

Improvement: Store customer details only in the Customers table and reference the customer using customer_id in the Orders table.

3.2 Restaurant Information

Restaurant names, addresses, and contact numbers are stored in the Restaurants table, while the MenuItems table references restaurants through restaurant_id.

If restaurant details were repeated for every menu item, changing a restaurant's address would require updating multiple records.

Improvement: Maintain restaurant details in the Restaurants table and use restaurant_id as a foreign key in the MenuItems table.

3.3 Menu Item Information

Food item names, descriptions, categories, and current prices are stored in the MenuItems table.

An order may contain several food items, and the same food item may appear in many orders. Repeating all menu item details in the Orders table would create unnecessary duplication.

Improvement: Use the OrderItems table to connect orders with menu items. Store the item_id as a foreign key and retain price_at_order to record the price charged when the order was placed.

3.4 Delivery Agent Information

Delivery agent names, phone numbers, and vehicle numbers are stored in the DeliveryAgents table. The Deliveries table references the assigned agent through agent_id.

Repeating the agent's contact information in every delivery record could create inconsistent information if the agent's phone number changes.

Improvement: Keep agent details in DeliveryAgents and use agent_id in Deliveries.

3.5 Total Order Amount

The Orders table contains total_amount, while the OrderItems table contains quantity and price_at_order.

The total order amount can generally be calculated by adding the quantities multiplied by their respective prices.

For example:

Item	Quantity	Price per Unit	Subtotal
Burger	2	₹120	₹240
French Fries	1	₹80	₹80
Total			**₹320**

The total_amount in Orders may duplicate information derivable from the order items if the total represents only the sum of their subtotals.

However, retaining the total can be useful for billing and historical records. It should be calculated or validated consistently, with any applicable taxes, discounts, delivery charges, and other adjustments accounted for.

Improvement: Define a consistent calculation rule for total_amount and ensure that it agrees with the order's item prices and applicable charges.

4. Functional Dependencies

A **functional dependency** describes a relationship in which one attribute or a group of attributes determines another attribute.

It is represented as:

X → Y

This means that the value of attribute X uniquely determines the value of attribute Y.

In the Online Food Delivery System, primary keys determine the other attributes associated with their respective records.

4.1 Functional Dependencies in the Customers Table

Attributes:

customer_id, full_name, email, phone_number, password_hash, registration_date

Functional dependency:

customer_id → full_name, email, phone_number, password_hash, registration_date

Explanation:

Each customer_id identifies one customer and determines that customer's name, email, phone number, password hash, and registration date.

Since email and phone number are marked UNIQUE in the existing design, they can also identify a customer when their values are present and unique.

4.2 Functional Dependencies in the Addresses Table

Attributes:

address_id, customer_id, address_line, city, state, pincode, address_type

Functional dependency:

address_id → customer_id, address_line, city, state, pincode, address_type

Explanation:

Each address_id identifies one delivery address and its associated customer.

A customer can have multiple addresses, so customer_id alone does not determine a single address.

4.3 Functional Dependencies in the Restaurants Table

Attributes:

restaurant_id, restaurant_name, cuisine_type, address, phone_number, rating, is_active

Functional dependency:

restaurant_id → restaurant_name, cuisine_type, address, phone_number, rating, is_active

Explanation:

Each restaurant_id identifies one restaurant and determines its stored details.

4.4 Functional Dependencies in the MenuItems Table

Attributes:

item_id, restaurant_id, item_name, description, price, category, is_available

Functional dependency:

item_id → restaurant_id, item_name, description, price, category, is_available

Explanation:

Each item_id identifies a menu item and determines its restaurant, name, description, price, category, and availability.

The restaurant may offer multiple menu items, so restaurant_id alone does not uniquely determine a particular menu item.

4.5 Functional Dependencies in the Orders Table

Attributes:

order_id, customer_id, restaurant_id, address_id, order_date, order_status, total_amount

Functional dependency:

order_id → customer_id, restaurant_id, address_id, order_date, order_status, total_amount

Explanation:

Each order_id identifies an order and determines the customer who placed it, the restaurant, the delivery address, the order date, the status, and the recorded total amount.

4.6 Functional Dependencies in the OrderItems Table

Attributes:

order_item_id, order_id, item_id, quantity, price_at_order

Functional dependency:

order_item_id → order_id, item_id, quantity, price_at_order

Explanation:

Each order_item_id identifies one order line and determines its order, menu item, quantity, and price at the time of ordering.

If the design permits a menu item to appear only once per order, the combination (order_id, item_id) can also be defined as a candidate key. In that case:

(order_id, item_id) → quantity, price_at_order

This additional dependency requires a corresponding uniqueness constraint and is not guaranteed by the current table structure alone.

4.7 Functional Dependencies in the DeliveryAgents Table

Attributes:

agent_id, agent_name, phone_number, vehicle_number, is_available

Functional dependency:

agent_id → agent_name, phone_number, vehicle_number, is_available

Explanation:

Each agent_id identifies one delivery agent and determines the agent's stored information.

4.8 Functional Dependencies in the Deliveries Table

Attributes:

delivery_id, order_id, agent_id, pickup_time, delivery_time, delivery_status

Functional dependency:

delivery_id → order_id, agent_id, pickup_time, delivery_time, delivery_status

Explanation:

Each delivery_id identifies a delivery record and determines the order, assigned agent, pickup time, delivery time, and delivery status.

The existing design specifies one delivery record per order. To enforce this relationship, order_id should also have a UNIQUE constraint in Deliveries.

4.9 Functional Dependencies in the Payments Table

Attributes:

payment_id, order_id, amount, payment_method, payment_status, payment_date

Functional dependency:

payment_id → order_id, amount, payment_method, payment_status, payment_date

Explanation:

Each payment_id identifies one payment record and determines its order, amount, method, status, and payment date.

The existing design specifies exactly one payment record per order. To enforce this relationship, order_id should have a UNIQUE constraint in Payments.

4.10 Functional Dependencies in the Reviews Table

Attributes:

review_id, order_id, customer_id, restaurant_rating, delivery_rating, comment, review_date

Functional dependency:

review_id → order_id, customer_id, restaurant_rating, delivery_rating, comment, review_date

Explanation:

Each review_id identifies a review and determines the customer, order, ratings, comment, and review date.

The existing design specifies at most one review per order. Therefore, order_id should have a UNIQUE constraint in Reviews.

Additionally, the database should ensure that the customer associated with a review is the customer who placed the corresponding order.

5. Insertion, Deletion, and Update Anomalies

Database anomalies are problems that arise when data is poorly organized or unnecessarily duplicated. The three main types are insertion anomalies, deletion anomalies, and update anomalies.

5.1 Insertion Anomaly

An insertion anomaly occurs when certain information cannot be added to the database without also inserting unrelated information.

Example 1: Adding a New Restaurant

Suppose a new restaurant joins the platform but has not added any menu items yet.

If restaurant details and menu items were stored in a single combined table, the restaurant might not be inserted until at least one menu item was available.

This would make it difficult to store restaurant information independently.

Example 2: Registering a Customer

If customer information were stored only alongside order details, a new customer could not be registered without creating an order.

Effect:

- New information may be delayed or impossible to insert.
- Unnecessary or incomplete records may be created.
- The database becomes difficult to maintain.

Solution: Store customers, restaurants, and menu items in separate tables, as in the existing database design. This allows a customer or restaurant to be registered independently of an order or menu item.

5.2 Deletion Anomaly

A deletion anomaly occurs when deleting one record unintentionally removes other important information.

Example 1: Deleting a Menu Item

Suppose restaurant details and menu items are stored in the same table. If the restaurant has only one menu item and that item is deleted, the restaurant's information may also be lost.

Example 2: Deleting an Order

If customer information is stored only in the Orders table, deleting a customer's last order could remove the only stored copy of that customer's details.

Effect:

- Important information may be lost unintentionally.
- Historical records may become unavailable.
- The database may no longer represent the real-world situation accurately.

Solution: Store customer and restaurant information separately from orders and menu items. Use foreign keys to maintain relationships and choose appropriate deletion rules. For example, deleting an order should not automatically delete the corresponding customer or restaurant.

5.3 Update Anomaly

An update anomaly occurs when the same information appears in multiple records and must be changed in every location.

Example 1: Changing a Restaurant Address

If a restaurant's address is repeated in every menu item record, changing the address requires updating multiple rows.

If even one row is missed, the database may contain conflicting addresses for the same restaurant.

Example 2: Changing a Customer's Phone Number

If a customer's phone number is repeated in several order records, changing the number requires updating all those records.

Effect:

- Different records may contain inconsistent information.
- Updates require additional time and effort.
- Errors become more likely as the database grows.

Solution: Store each customer's details in Customers and each restaurant's details in Restaurants. Reference these records using foreign keys instead of repeating their details in related tables.

6. How Poor Database Design Leads to Anomalies

Poor database design often results from combining multiple entities into a single table, storing the same facts repeatedly, or failing to define appropriate keys and relationships.

For example, consider a poorly designed table:

Order_ID	Customer_Name	Customer_Phone	Restaurant_Name	Item_Name	Quantity
101	Rahul	9876543210	Food Palace	Burger	2
101	Rahul	9876543210	Food Palace	Fries	1
102	Rahul	9876543210	Food Palace	Pizza	1
103	Anu	9123456780	Spice Hub	Dosa	2

This table repeats customer and restaurant information whenever the same customer or restaurant appears in another order.

The design creates the following problems:

1. **Redundancy:** Rahul's name and phone number are repeated in several rows.
2. **Insertion anomaly:** A new restaurant cannot be stored naturally without an order and menu item if the table requires all those details.
3. **Deletion anomaly:** Deleting the only order for a restaurant may also remove its only stored restaurant details.
4. **Update anomaly:** Changing Rahul's phone number requires updating every row containing his details.
5. **Data inconsistency:** If one row is not updated, the same customer may appear with different phone numbers.

The improved design separates this information into Customers, Restaurants, MenuItems, Orders, and OrderItems.

This makes each table responsible for a specific entity or relationship and reduces unnecessary repetition.

7. Proposed Improvements to the Database Design

The existing database is already organized into separate tables, which is a good foundation. The following improvements can make it more reliable and maintainable.

7.1 Apply Database Normalization

Normalization is the process of organizing data into tables to reduce unnecessary redundancy and prevent anomalies.

First Normal Form (1NF)

A table satisfies 1NF when each field contains a single value and there are no repeating groups.

The existing design supports 1NF because multiple delivery addresses are stored in Addresses, and multiple food items in an order are stored as separate OrderItems records.

Improvement: Continue storing individual addresses and order items in separate rows instead of placing multiple values in a single field.

Second Normal Form (2NF)

A table satisfies 2NF when it is in 1NF and every non-key attribute depends on the whole of each candidate key, with no partial dependency on part of a composite candidate key.

The current tables use individual primary keys. If a composite key such as (order_id, item_id) is used for OrderItems, the attributes must depend on the entire key.

For example, quantity and price at ordering depend on the specific menu item within a particular order, not on the order or menu item independently.

Improvement: Use appropriate primary keys and ensure that attributes describe the complete record represented by the key.

Third Normal Form (3NF)

A table satisfies 3NF when it is in 2NF and has no inappropriate transitive dependencies among non-key attributes.

For example, customer contact details should depend on customer_id, not on order_id through duplicated customer information.

Improvement: Keep customer details in Customers, restaurant details in Restaurants, and delivery agent details in DeliveryAgents. Reference these entities using foreign keys.

The existing design follows the principles of normalization. However, confirming that every table satisfies a normal form requires checking all candidate keys and functional dependencies.

7.2 Strengthen Primary and Foreign Key Constraints

Primary keys and foreign keys maintain entity integrity and referential integrity.

Recommended constraints include:

- Every table should have a primary key.
- Addresses.customer_id should reference Customers.customer_id.
- MenuItems.restaurant_id should reference Restaurants.restaurant_id.
- Orders.customer_id should reference Customers.customer_id.

- Orders.restaurant_id should reference Restaurants.restaurant_id.
- Orders.address_id should reference Addresses.address_id.
- OrderItems.order_id should reference Orders.order_id.
- OrderItems.item_id should reference MenuItems.item_id.
- Deliveries.agent_id should reference DeliveryAgents.agent_id.
- Payments.order_id should reference Orders.order_id.
- Reviews.order_id and Reviews.customer_id should reference their corresponding parent records.

These constraints prevent invalid references and help maintain consistent relationships.

7.3 Add Appropriate UNIQUE Constraints

The existing design describes one delivery, one payment, and one review per order.

To enforce these relationships, the following constraints should be added:

- Deliveries.order_id — UNIQUE
- Payments.order_id — UNIQUE
- Reviews.order_id — UNIQUE

The existing Customers table already specifies UNIQUE constraints on email and phone number.

These constraints should match the actual business rules of the application.

7.4 Add Validation Constraints

Validation constraints prevent invalid data from entering the database.

Recommended rules include:

- Menu item prices must not be negative.
- Order item quantities must be greater than zero.
- Payment amounts must not be negative.
- Restaurant ratings and review ratings must be within the range of 1 to 5.
- Order status must contain an allowed value.
- Payment status must contain an allowed value.
- Delivery status must contain an allowed value.

For example:

CHECK (quantity > 0)

This constraint ensures that an order item has a positive quantity.

Similarly:

CHECK (restaurant_rating BETWEEN 1 AND 5)

This ensures that a restaurant rating is within the permitted range.

7.5 Preserve Historical Order Prices

The MenuItems table stores the current price of each food item, while OrderItems stores price_at_order.

This is an important feature because menu prices may change over time.

For example, if a burger costs ₹120 when an order is placed and its price later increases to ₹150, the historical order should still reflect the original price of ₹120.

Improvement: Retain price_at_order in OrderItems and use it when calculating historical order totals.

7.6 Ensure Address Ownership

The Orders table stores both customer_id and address_id.

The database should ensure that the selected delivery address belongs to the customer who placed the order.

Otherwise, an order could accidentally reference another customer's address.

Improvement: Validate address ownership through application logic or an appropriate database constraint. One possible relational design is to enforce a composite foreign key using (customer_id, address_id) referencing a matching unique pair in Addresses.

7.7 Maintain Consistent Order Totals

The total order amount should correspond to the order's item quantities, historical prices, and any applicable additional charges or discounts.

Improvement: Define a single calculation rule and ensure that the stored total is updated or validated whenever order details change.

7.8 Use Appropriate Deletion Rules

Deleting a record should not unintentionally remove unrelated or historically important information.

For example, deleting a customer should not automatically delete financial records associated with completed orders.

Improvement: Use suitable foreign-key deletion rules, restrict the deletion of referenced records when necessary, and consider deactivating restaurants or delivery agents rather than deleting records that are still needed for historical reporting.

8. Summary of Proposed Improvements

Issue	Possible Problem	Proposed Improvement
Repeated customer information	Update anomalies	Store customer details only in Customers.
Repeated restaurant information	Redundancy and inconsistency	Store restaurant details only in Restaurants.
Multiple items in an order	Repeating groups	Use the OrderItems linking table.
Missing key constraints	Invalid or duplicate records	Apply primary, foreign, and UNIQUE constraints.
Invalid prices or quantities	Incorrect order information	Add CHECK constraints.
Menu price changes	Incorrect historical totals	Preserve price_at_order.
Incorrect address references	Delivery to an unintended address	Validate that the address belongs to the customer.
Uncontrolled deletions	Loss of historical data	Apply suitable deletion rules.
Incorrect order totals	Billing inconsistencies	Define and validate a consistent total calculation.

9. Conclusion

The analysis of the Online Food Delivery System demonstrates the importance of proper database design. The original design contains ten related tables, each responsible for storing a particular type of information or relationship.

Separating Customers, Restaurants, MenuItems, Orders, OrderItems, DeliveryAgents, Deliveries, Payments, and Reviews helps reduce unnecessary data redundancy. Functional dependencies show how primary keys determine the remaining attributes in each table.

Poorly designed databases can suffer from insertion, deletion, and update anomalies. These problems may cause data loss, duplication, and inconsistencies. Normalization, appropriate keys, validation constraints, and carefully defined relationships help prevent such problems.

The proposed improvements strengthen the existing design without changing its overall purpose. They also improve data integrity, preserve historical order information, and make the database easier to maintain.

Therefore, a well-designed relational database is essential for ensuring the accuracy, consistency, reliability, and efficiency of an online food delivery application.